Metric-Grounded Analytics Agents: Measuring Semantic Reliability Beyond Executable SQL

Your Name*
New York University

Abstract

Natural-language analytics systems can produce executable SQL while computing the wrong measure, grain, grouping, or population. We present a research prototype for evaluating this gap on six official U.S. Centers for Medicare & Medicaid Services (CMS) Plan Year 2026 Marketplace public-use files. The system separates metric routing, constrained query construction, static policy validation, database execution, evidence citation, and result comparison. A 30-question benchmark evaluates a question-only Codex condition, an oracle-table-conditioned Codex-to-SQL pilot, a benchmark-specific oracle-routed deterministic compiler, and lexical and Codex metric routers. We report strict complete-row agreement separately from a more permissive alias/projection-compatible score, together with Top-k, group, numeric, execution, and traceability measures. In the original pilot, all 90 Codex-generated queries executed, but only 3.3% strictly matched reference results; 18.9% matched under compatible projection. The oracle compiler achieved 80.0% strict and 100% projection-compatible agreement, but this is an upper bound on a co-developed benchmark, not learned generalization. On a hash-locked, model-generated lexical-shift challenge, strict agreement fell to 40.0% even with oracle metric routing. In this benchmark, these results show that execution is a weak proxy for semantics and that metric routing alone does not solve query generalization. A pinned API-model rerun (DeepSeek) replicates the execution–semantics gap—21–38% executable SQL but 0% strict complete-result agreement—and shows that an injected metric registry nearly doubles executability and lowers numeric error without reaching strict correctness. The gap replicates again on a second, non-healthcare domain (NYC 311 Service Requests) with a third model family (Claude): 96.7% executable SQL yet still 0% strict agreement (bootstrap interval $[0.000, 0.000]$), and Claude reproduces the same 0% strict result on the original CMS benchmark, giving a same-domain three-model comparison. A deterministic compiler ported to the second domain reaches 0.900 strict, so the questions are answerable and the 0% is an LLM-compilation gap, not an artifact of one benchmark or one model. The study is a manuscript-in-preparation diagnostic; independent human-authored testing and qualitative review remain pending.

1 Introduction

Large language models have made database interfaces easier to build, but a successful tool call is not the same as a correct analytical answer. A query may run without error while using the wrong denominator, counting rows rather than entities, comparing unmatched populations, or returning the wrong output grain. The resulting prose can remain plausible because neither fluency nor SQL execution verifies the intended metric.

This problem is especially visible in health-insurance marketplace data. Premium rates, plans, service areas, benefits, crosswalks, and quality ratings are published at different grains. "Plan count,"

*Fill in author name and affiliation.

"average premium," "issuer competition," and "quality versus premium" therefore require explicit choices about keys, filters, joins, and aggregation. A natural-language interface that hides those choices can make an incorrect result look authoritative.

We study a narrow research question:

> When analytics questions are routed through an explicit metric registry and a constrained compiler, which intermediate failures can be localized, and which semantic errors remain?

The goal is not to establish general healthcare reasoning or universal hallucination reduction. Instead, the project contributes an inspectable evaluation scaffold and a set of negative findings about executable SQL, oracle routing, and compiler generalization.

The work makes five contributions:

1. A reproducible DuckDB/dbt research substrate over six official CMS Marketplace files.
2. A 30-question domain benchmark with reference SQL stored outside runtime agent paths.
3. An evaluation that separates routing, compilation, execution, strict result agreement, compatible projection, evidence, and abstention.
4. A conservative empirical analysis that distinguishes subscription-Codex pilots, oracle upper bounds, predicted routing, post-hoc repairs, and locked model-generated diagnostics.
5. A cross-domain, cross-model replication (NYC 311 Service Requests; Claude) showing the execution–semantics gap is not specific to one benchmark or model.

2 Related work

Spider established cross-domain text-to-SQL evaluation across database splits [1]. BIRD extended this direction to larger databases, real content, and domain-knowledge requirements [2]. Work on semantic evaluation has shown that SQL string match is an incomplete correctness measure: different queries can be equivalent, and similar-looking queries can differ semantically [3, 4]. Our work therefore compares executed result tables rather than generated SQL strings.

Execution-based evaluation is also insufficient by itself. A syntactically valid query can execute against the wrong table, grain, or population. Test-suite methods help distinguish some semantically different queries [3], but analytics questions additionally depend on operational metric definitions. This motivates an explicit semantic registry rather than relying only on database schema.

Grounded generation and retrieval-augmented generation distinguish parametric model output from external evidence [5]. In analytics, however, retrieved text does not specify a denominator or prove that a number was computed correctly. We treat metric definitions and executed result rows as different evidence types: the registry constrains what should be computed, while result rows record what was actually returned.

Our prototype is closest to semantic-layer-assisted analytics agents, but the present deterministic compiler is hand-authored. It should be interpreted as a benchmark-specific upper bound and diagnostic instrument, not a learned semantic parser.

3 Data and warehouse

3.1 CMS Marketplace public-use files

The study uses official CMS Plan Year 2026 public-use files for states using the HealthCare.gov platform [6, 7]. The validated local build contains:

Source	Raw rows
Rate PUF	2,235,761
Benefits and Cost Sharing PUF	1,457,952
Plan Attributes PUF	22,059
Service Area PUF	8,820
Plan ID Crosswalk, PY2025–PY2026	158,746
Quality PUF	4,302

These files describe public plan offerings, published rates, benefit design, service areas, plan continuity, and public quality ratings. They do not contain claims, enrollment, subsidy-adjusted consumer prices, utilization, individual outcomes, or patient records. No causal inference is attempted.

3.2 Analytics substrate

DuckDB provides local execution and dbt defines staging, intermediate, dimensional, and fact models. The current project contains 20 models and 97 data tests covering key uniqueness, required values, accepted categories, and fact-to-dimension relationships. The complete `dbt build` contains 120 nodes: 20 models, 97 data tests, one seed, and two no-op exposures.

The modeled entities include plans, issuers, geography, metal levels, benefits, age bands, premium observations, plan availability, benefit cost sharing, plan history, and quality ratings. Raw files and the local database are excluded from source control; compact provenance manifests record source and artifact hashes.

4 Research design

4.1 Questions

The benchmark contains 30 questions across premiums, issuer competition, plan availability, benefit coverage, plan continuity, plan design, and quality. Each question records difficulty, category, approved source tables, one or more metric labels, and a reference SQL path. Reference SQL is executed separately to create gold result JSON.

The benchmark and compiler were co-developed. Consequently, performance on the 30 source questions estimates implementation consistency, not generalization. Oracle metric labels are an explicit upper bound.

4.2 Diagnostic evaluation sets

Two transformations probe sensitivity to wording:

- **Close paraphrases:** two Codex-generated rewrites per source question, 60 total. These remain lexically close and are used for development diagnostics.
- **High-shift challenge:** one Codex-generated rewrite per question, 30 total, with a frozen content hash. The prompt requests lexical and syntactic distance without changing the measure, filters, grouping, or direction.

The challenge was not used for further compiler repair. Nevertheless, both sets inherit labels and reference rows from their source questions and have not been independently validated by domain experts. We therefore do not call them human-authored confirmatory tests.

4.3 Research questions

- **RQ1—Execution:** How often does each system return a response and, where applicable, executable SQL?
- **RQ2—Strict semantics:** How often does a successful run reproduce the full reference schema and rows?
- **RQ3—Partial semantics:** When strict equality fails, how much group, Top-k, and numeric agreement remains?
- **RQ4—Routing and compilation:** How much error is attributable to metric routing versus deterministic compilation?
- **RQ5—Traceability:** Which systems expose metric, table, SQL, validation, and result-row evidence?

Qualitative unsupported-claim and caveat-quality hypotheses are deferred until two independent reviewers complete the prepared system-label-blinded packet.

5 Systems

5.1 Direct Codex

Direct Codex receives only the natural-language question and is prohibited from using tools, files, databases, code execution, or the internet. Because the questions require empirical CMS data, answers without execution are classified as explicit abstentions or unverified responses. Direct output is not assigned a result-match score.

5.2 Subscription Codex-to-SQL

Codex-to-SQL receives the warehouse schema and a question-specific list of permitted tables and required terms. It returns one read-only DuckDB query, which is checked by a static table/statement policy and then executed. This condition is stronger than question-plus-schema text-to-SQL because table routing is supplied. It is therefore described as **oracle-table-conditioned Codex-to-SQL**.

The implementation launches each batch in an ephemeral empty working directory, ignores repository rules and user configuration, disables tool/data access, and captures the structured Codex output. It does not receive gold SQL, gold rows, or metric identifiers.

5.3 Oracle-routed deterministic compiler

Benchmark metadata supplies approved metric slugs. A metric registry defines expressions, permitted tables, dimensions, and caveats. A hand-authored compiler uses the metric set and question wording to choose a query template. SQL is validated and executed, and the record includes metric definitions, validation checks, table references, result rows, and citations.

No runtime module reads `benchmark/gold_sql/` or `benchmark/gold_answers/`. However, the compiler contains benchmark-specific branches and was developed alongside the questions. Oracle results must not be interpreted as an unbiased learned-agent comparison.

5.4 Predicted metric routing

The lexical router combines word, word-bigram, and character n-gram cosine similarity over source questions and metric-registry descriptions. Its multi-label relative threshold is selected by leave-one-source-question-out calibration; paraphrases and challenge questions are not used for threshold selection.

The Codex router receives metric names, expressions, dimensions, caveats, and held-out question text, then selects the smallest metric-slug set. Three independent batched calls measure stochastic stability. Both routers feed the same deterministic compiler, isolating route errors from compiler errors.

5.5 Registry-grounded LLM SQL (grounding ablation)

The systems above bound the problem from two sides—an LLM with no metric grounding (`llm_sql`) and a hand-authored compiler with full grounding (`metric_grounded`)—but leave the middle unmeasured: an LLM given the metric registry. We therefore add `llm_registry_sql`, which is identical to the schema-only `llm_sql` baseline in model, task, schema context, static validation policy, and permitted-table gate, and differs only by injecting the benchmark's oracle metric definitions (expression, primary tables, allowed dimensions, and caveats) into the prompt. Because the metric labels come from the benchmark, this is an oracle-metric-conditioned generator, directly comparable to the oracle-routed compiler; because it still compiles its own SQL, its paired strict-match difference from `llm_sql` estimates the causal effect of supplying an explicit registry to a learned generator while holding the compiler absent. This condition is defined here and is reported under the pinned, snapshot-identified API rerun described in the reproducibility runbook; it is not included in the earlier subscription-Codex figures below.

6 Evaluation

6.1 Strict and compatible result agreement

The primary **end-to-end strict result match** requires successful execution, identical output column names, identical row cardinality, and one-to-one equality of every row and cell within a numeric tolerance. Row ordering is ignored in the strict table-set measure because ties can be nondeterministic; ranking is evaluated separately. **Conditional strict match** uses only executed runs as the denominator.

We retain **compatible projection match** as a secondary diagnostic. It aligns dimension and numeric columns by matching names and then compatible type/order, allowing harmless alias or auxiliary-column differences. This was the project's original "exact" score; it is no longer described as strict complete-result agreement.

6.2 Partial agreement and numeric error

For grouped outputs, we report group precision, recall, F1, and Jaccard. For ranked outputs, we report Top-k precision, recall, and Jaccard using the gold k as the recall denominator, so short outputs are penalized. Numeric metrics include mean absolute error, relative error, and bounded symmetric mean absolute percentage error (SMAPE). Missing required strict columns cause strict failure; they are not silently accepted.

6.3 Evidence and failure reporting

Execution success, static SQL policy compliance, router abstention, missing credentials, missing databases, model errors, SQL errors, prompt, SQL, result rows, latency, token use, and failure class are recorded separately. Static policy validation is not called executable SQL validity.

Numeric-claim evidence presence checks whether numbers in generated prose occur in executed rows. It does not judge qualitative entailment. The automated support status is not used to claim lower unsupported-claim rates before human review.

6.4 Repeats and uncertainty

Stochastic Codex conditions run three times. Deterministic systems need only one independent execution; retained technical repeats are identified as such. Bootstrap estimates first average repeats within `question_id`, then resample question IDs with replacement 10,000 times. Reported intervals therefore contain 30 question clusters rather than treating 90 records as independent.

7 Results

7.1 Original benchmark: execution versus strict semantics

Table 1 reports the completed subscription-Codex pilot and deterministic oracle compiler after rescoring saved rows with the stricter 2026-07-13 contract.

Condition	Runs	Exec.	Strict	Compat.	Top-k J.	Group F1	SMAPE
Oracle-table Codex-to-SQL	90	1.000	0.033	0.189	0.658	0.613	0.364
Oracle compiler	90†	1.000	0.800	1.000	1.000	1.000	0.000

Table 1: Original benchmark. †technical records.

All 90 Codex SQL queries executed, but only three strictly reproduced the full reference schema and rows. Seventeen matched under compatible projection. This large execution–semantics gap directly supports the project's measurement motivation.

The oracle compiler strictly matched 24 of 30 unique source questions. Its 100% compatible projection score reflects correct core dimensions/measures with some auxiliary-column differences; it is not a 100% strict result. With 10,000 question-clustered resamples, strict agreement was 0.033 (95% interval 0.000–0.100) for Codex-to-SQL and 0.800 (0.667–0.933) for the oracle compiler. The paired difference was 0.767 (0.600–0.900).

This is not a fair estimate of general model superiority: the compiler receives oracle metric labels and contains benchmark-specific logic. The comparison is useful as an upper-bound ablation showing what explicit semantic contracts can make inspectable.

7.2 Direct Codex pilot

Direct and SQL conditions were isolated in six calls: 30 questions, three repeats, and two conditions. Direct Codex returned all 90 responses. Sixty explicitly stated that the requested empirical result could not be determined without data. The remaining 30 were retained as unverified rather than scored against result rows. Direct Codex used 36,277 input and 2,477 output tokens.

Only one source question produced identical Codex-generated SQL across all three repeats. Twenty-seven produced three distinct queries and two produced two. SQL variation persisted even though every query executed.

The Codex CLI reported a default logged-in model rather than a stable public snapshot. System instructions were present, usage and latency were batch-level, and subscription USD cost was unavailable. These results are not interchangeable with the planned raw API experiment.

7.3 Metric-routing accuracy

On 60 close paraphrases, the calibrated lexical router achieved 0.867 exact metric-set accuracy, 0.950 Top-1 accuracy, 0.950 macro precision, and 0.908 macro recall. Leave-one-source-question-out exact accuracy was only 0.533, indicating that close-paraphrase performance is optimistic for broader generalization.

Across 180 Codex routing records, exact metric-set accuracy was 0.833, Top-1 accuracy 0.867, macro precision 0.867, and macro recall 0.850. Predictions were identical across repeats for 54 of 60 examples. Per-repeat exact accuracy was 0.867, 0.833, and 0.800. The deterministic lexical router had higher observed agreement than the Codex router on this benchmark-specific close robustness set; the conditions are not a general model comparison.

7.4 Post-hoc compiler development

The first end-to-end close-paraphrase run exposed brittle compiler phrase matches and incomplete table contracts. Before repair, compatible-projection agreement was 0.517 with oracle routing and 0.450 with lexical routing. Compiler rules were then broadened after inspecting these failures.

After repair and strict rescoring, oracle-route strict agreement was 0.800 and lexical-route strict agreement 0.733. Their compatible-projection scores were 1.000 and 0.917. Repeated Codex routing achieved 0.667 end-to-end strict match, 0.769 conditional strict match among executed runs, and 24/180 explicit router abstentions.

Because the repairs used observed paraphrase failures, all values in this section are development results. They are not presented as unbiased held-out evidence.

7.5 Hash-locked high-shift diagnostic

The higher-shift challenge was generated and hash-locked before its evaluation, and no further compiler repair used its results. Table 2 reports strict rescoring.

Route/system	Runs	Exec.	Strict	Compat.	Top-k J.	Group F1	SMAPE
Oracle metric route	30	1.000	0.400	0.567	0.652	0.670	0.230
Lexical predicted route	30	1.000	0.300	0.467	0.574	0.607	0.288
Codex predicted route	90	0.844	0.378	0.645	0.685	0.702	0.219
Oracle-table Codex-to-SQL	90	0.967	0.000	0.138	0.659	0.583	0.545

Table 2: Hash-locked high-shift challenge (conditional compatible projection).

Even with correct metric labels, the deterministic compiler strictly matched only 40.0% of questions. This identifies query compilation and output-contract generalization as a major failure source. Codex routing plus compilation achieved 37.8% end-to-end strict match, while no challenge Codex-to-SQL run reproduced the full reference schema and rows; 13.8% of executed runs matched under compatible projection.

Question-clustered strict agreement intervals were 0.233–0.567 for oracle routing, 0.133–0.467 for lexical routing, and 0.211–0.556 for Codex routing. The paired end-to-end strict difference between Codex routing plus compilation and Codex-to-SQL was 0.378 (0.211–0.544). The Codex-routed versus lexical-routed end-to-end strict difference was 0.078 with an interval crossing zero (−0.022–0.200).

The challenge remains model-generated and inherits labels from source questions. It is a stronger locked diagnostic, not an independently human-authored test.

7.6 Failure taxonomy

Separating the pipeline into inspectable stages lets each failing run be assigned to the earliest stage that broke rather than to an undifferentiated "wrong answer." Across the saved Codex-to-SQL and predicted-routing records, the observed failures fall into five classes. Counts are illustrative of the saved diagnostic runs, not population estimates.

Failure class	Localized in	Signature	Representative cases
Metric-route mismatch	routing	predicted metric set ≠ oracle set; compiler computes the wrong measure	Q003_P2 (premium routed to metal-difference); Q027 quality+deductible collapsed to deductible
Output-contract mismatch	compilation	correct measure and grain but wrong columns, cardinality, or missing rows under strict scoring	strict-vs-compatible gap on the original benchmark (3.3% vs 18.9%)
Ranking/Top-k error	compilation	correct group set but wrong order or truncated k	low Top-k Jaccard on Q008, Q009, Q023 while group match stays higher
Grouping error	compilation	wrong partition or filter yields a different group set	Q016, Q018, Q026 group-match collapse
Numeric-value error	compilation/execution	executable rows whose numbers diverge from gold (high SMAPE)	Q017, Q022, Q010 SMAPE ≥ 0.9 despite successful execution

Table 3: Failure taxonomy over saved diagnostic runs.

Two patterns are load-bearing for the paper's argument. First, execution success never appears as a discriminating signal: every class above contains runs whose SQL executed cleanly. Second, the failures concentrate in **compilation and output-contract** stages rather than routing—on the locked challenge, oracle routing eliminates route error by construction yet still leaves 60% strict failure. A pipeline that reported only router accuracy or execution rate would mislabel most of these runs as successes.

7.7 Pinned API-model rerun (DeepSeek)

To retire the model-identity threat for the online conditions, we reran the full benchmark against a pinned, snapshot-identified API model reachable without a subscription CLI. The runtime requested `deepseek-chat` and the service reported `deepseek-v4-flash`; each call records the served model, token usage, and an estimated cost. This run evaluates four conditions—direct (question only), schema-only LLM-SQL, LLM-SQL plus the injected metric registry (§5.5), and the oracle compiler—at three repeats over the 30 questions (90 runs each). Table 4 reports the strict-rescored summary with 10,000-sample question-clustered bootstrap intervals.

Three observations replicate and sharpen the subscription-pilot findings with a citable model. First, the execution–semantics gap persists: the LLM-SQL systems produced executable SQL on 21–38% of runs but reproduced the complete reference result on **0%** (bootstrap interval $[0.000, 0.000]$);

Condition	Exec. SQL	Strict	Compat. proj.	Numeric SMAPE
Direct (no data access)	—	0.000	0.000	—
Schema-only LLM-SQL	0.211	0.000 [0.000, 0.000]	0.033	0.676 [0.322, 0.934]
LLM-SQL + registry	0.378	0.000 [0.000, 0.000]	0.044	0.356 [0.033, 0.722]
Oracle compiler	1.000	0.800 [0.667, 0.933]	1.000	0.000 [0.000, 0.000]

Table 4: Pinned DeepSeek rerun with question-clustered bootstrap intervals.

direct answers were 100% unsupported. Second, injecting the metric registry (schema-only → registry) nearly doubled the executable-SQL rate (0.211 → 0.378) and lowered numeric error, with a paired registry-minus-schema-only SMAPE difference of −0.407 (95% interval [−0.814, 0.000])—an improvement whose interval just reaches zero. The same paired comparison showed no significant difference on the questions both systems executed for Top-k overlap (−0.019 [−0.100, 0.066]) or group match (0.005 [−0.056, 0.081]); the lower marginal Top-k/group means for the registry condition reflect its larger and harder executed set, not a regression. Third, neither LLM condition reached strict correctness, while the oracle compiler again scored 0.800 [0.667, 0.933]. Metric grounding therefore improves executability and numeric fidelity for a real API model but does not, by itself, deliver strict complete-result correctness—consistent with locating the residual failure in query compilation and output-contract generation. The subscription-Codex figures in §7.1 and §7.5 are retained as earlier diagnostics; this run is the snapshot-identified replacement the model-identity threat called for. Section 7.8 extends it to a second domain and a third model family.

7.8 Cross-domain, cross-model replication (NYC 311, Claude)

To test whether the execution–semantics gap is specific to the CMS benchmark or to a single model, we replicated the two LLM conditions on an independent, non-healthcare domain with a different model family. The domain is NYC 311 Service Requests (full-year 2024; 3,456,770 requests) built into a DuckDB marts schema mirroring the CMS layout. Its metric registry deliberately encodes contested definitions—response time over closed requests only, which statuses count as resolved, per-capita normalization requiring a borough-population join, and window-function share denominators—so that a schema-only model can diverge from the registry-grounded condition. We built a 30-question benchmark (10 simple / 12 intermediate / 8 hard) with reference SQL verified to execute and return non-empty results against the warehouse. The model is Anthropic `claude-haiku-4-5-20251001`, a dated snapshot, run at three repeats over the 30 questions (90 runs each). We also ported the oracle-routed deterministic compiler to this domain—an independent rule-based generator authored from the NYC metric registry, not from the gold SQL—to establish the second-domain upper bound. Table 5 reports the strict-rescored summary with 10,000-sample question-clustered bootstrap intervals.

Condition	Exec. SQL	Strict	Compat. proj.	Numeric SMAPE
Schema-only LLM-SQL	0.967	0.000 [0.000, 0.000]	0.207	0.551 [0.265, 0.865]
LLM-SQL + registry	0.967	0.000 [0.000, 0.000]	0.241	0.397 [0.123, 0.723]
Oracle compiler	1.000	0.900 [0.800, 1.000]	0.933	0.000 [0.000, 0.000]

Table 5: NYC 311 second domain with Claude and a ported oracle compiler.

The core finding replicates across both domain and model family. Despite a high executable-SQL rate of 96.7%—far above the 21–38% seen with DeepSeek on the CMS questions—strict complete-

result agreement is again **0%**, with a bootstrap interval of $[0.000, 0.000]$ for both conditions. A 97%-executable model that reproduces the exact reference table on none of 90 runs is the sharpest form of the execution–semantics gap: fluency and execution success are uninformative about strict correctness. The registry again helps directionally without reaching significance or strict correctness. It lowered numeric error (SMAPE 0.551 → 0.397; paired registry-minus-schema-only -0.154, 95% interval $[-0.460, 0.171]$) and, most strikingly, cut the mean numeric relative error from 25,958 to 4.66—the signature of the per-capita denominator trap, where the schema-only model chooses a wrong normalization and returns ratios off by orders of magnitude that the registry definition corrects. Compatible projection (0.207 → 0.241), Top-k overlap (paired 0.018 $[-0.051, 0.104]$), group match (0.032 $[-0.000, 0.097]$), and traceability (0.495 → 0.745) all moved in the registry's favor, with paired intervals that reach zero. As on the CMS domain, metric grounding improves executability-adjacent fidelity and evidence but does not deliver strict complete-result correctness, locating the residual failure in query compilation and output-contract generation rather than in metric grounding. The second-domain oracle compiler makes the same point from the other side: a deterministic generator authored from the registry reached 0.900 strict $[0.800, 1.000]$ on the identical questions, so the questions are answerable with correct compilation—the 0% is an LLM-compilation gap, not an artifact of impossible questions. The three strict misses were two date-serialization mismatches and one projection-naming difference, mirroring the CMS oracle's strict-versus-compatible margin. With two domains and three model families (subscription Codex, DeepSeek, Claude) all showing a 0% LLM strict interval of $[0.000, 0.000]$ against oracle upper bounds of 0.80–0.90, the gap is not an artifact of one benchmark or one model.

7.9 Third model on the CMS benchmark (Claude)

To complete a same-domain three-model comparison, we also ran the two LLM conditions on the original CMS benchmark with `claude-haiku-4-5-20251001` (three repeats, 90 runs each). Table 6 places all three snapshot-identified models on the identical questions.

CMS, schema-only LLM-SQL	Exec. SQL	End-to-end strict
Subscription Codex (§7.1 pilot)	1.000	0.033
DeepSeek (§7.7)	0.211	0.000 [0.000, 0.000]
Claude (this section)	0.500	0.000 [0.000, 0.000]

Table 6: Same-domain three-model comparison on CMS.

Strict complete-result agreement is again **0%** for both Claude conditions (schema-only and registry-grounded), so the execution–semantics gap holds for a third model on the original domain. Two nuances are worth stating honestly. First, Claude's executable rate (0.500 schema-only, 0.489 registry) is depressed not by SQL errors but by the benchmark's strict source-table allow-list: 42 of 45 schema-only failures were validation rejections (`ValueError`) for joining a descriptive dimension table the question did not pre-declare—typically `dim_geography` for a state name or `dim_metal_level` as a proper dimension—while only three were execution errors. Executability is therefore itself sensitive to the validation policy and to how much descriptive structure a model volunteers, whereas strict correctness is robustly zero regardless. Second, the registry did **not** help here and slightly hurt: paired registry-minus-schema-only Top-k was -0.124 (95% interval $[-0.278, -0.014]$, excluding zero) and group match -0.093 ($[-0.257, 0.004]$), because the registry's listed primary tables encouraged still more non-declared joins and thus more allow-list rejections. This contrasts with DeepSeek on CMS (registry nearly doubled executability) and Claude on NYC 311 (registry halved numeric error):

metric grounding's effect on the executed set is model- and harness-dependent and is not uniformly positive. The one invariant across every model, domain, and grounding condition is the 0% strict interval.

8 Discussion

8.1 Execution is not semantic correctness

The most robust finding is the difference between execution and strict result agreement. Every original Codex SQL query executed, yet only 3.3% reproduced the full reference result. On the lexical-shift challenge, execution remained 96.7% while strict agreement was zero. Static table policies and database execution are necessary safety gates, but they do not establish analytical meaning.

8.2 Metric routing is only one layer

Correct metric routing did not guarantee correct rows. Oracle routing on the challenge achieved 100% route accuracy by definition but only 40% strict result agreement. This decomposition is important: in this locked diagnostic, research that reports router labels without executing the downstream query would miss a major observed error source.

8.3 Strict and compatible metrics answer different questions

Strict schema-and-row equality is conservative and penalizes harmless alias or auxiliary-column differences. Compatible projection captures semantic overlap but can accept incomplete outputs or incorrect positional mappings if used as a primary exact score. Reporting both exposes this trade-off. Future work should replace heuristic compatible mapping with independently specified per-question output contracts and semantic alias dictionaries.

8.4 Traceability is a defensible contribution

The oracle/predicted compiler records metric definitions, allowed tables, SQL, policy checks, executed rows, and citations. This does not guarantee correctness, but it creates inspectable intermediate objects that help locate a failure in routing, compilation, validation, execution, or prose. The project's narrow contribution is therefore measurement and auditability, not solved reasoning.

9 Threats to validity

Benchmark co-development. The questions, reference SQL, metric registry, and deterministic compiler were developed in the same project. Oracle results are upper bounds and can encode benchmark-specific knowledge without directly reading gold files. As a first check on the reference answers themselves, every CMS gold query was audited by execution against the built warehouse (all 30 return non-empty results at the metric-registry grain) and given a documented human sign-off; two questions whose gold grain was finer than the question wording (quality-status distribution, crosswalk reasons) were tightened to match the question, and five defensible modeling choices were recorded rather than changed.

Test-set independence. Both evaluation sets were generated by Codex from the source questions. The same subscription interface was later evaluated as a router and SQL generator. No independent domain expert has yet confirmed every rewrite's intended metric, grain, filter, and output contract.

Model identity. Subscription Codex exposes CLI version and batch usage but not a stable underlying model snapshot. Reproduction at a later date may use a different default model.

Baseline conditioning. The Codex-to-SQL pilot receives question-specific allowed tables and required terms. It is not a pure question-plus-schema baseline.

Measurement. Strict equality can under-credit semantically equivalent aliases; compatible projection can over-credit incomplete or positionally aligned outputs. Both are reported. Numeric measures cover only comparable numeric cells and do not evaluate qualitative claims.

Human evaluation. The packet is system-label blinded rather than fully condition blinded because SQL/evidence presentation can reveal system family. Two independent reviews and adjudication remain incomplete, so no qualitative faithfulness result is claimed.

Statistical scope. Each bootstrap resamples 30 questions within a single domain; the study now spans two domains (CMS, NYC 311) and three model families, but no interval represents uncertainty jointly over schemas, domains, model snapshots, or human-authored intents. The cross-domain replication (§7.8) strengthens external validity qualitatively rather than through a pooled random-effects estimate.

External validity. CMS Marketplace data is public insurance-market data, not clinical or claims data. Findings should not be transferred to medical decision support.

10 Reproducibility and ethics

The repository includes exact commands for data download, validation, DuckDB loading, dbt build, gold generation, all evaluation conditions, rescoring, question-clustered bootstrap, blind-review export, and CI. Compact artifacts record source hashes, code state, and summary outputs; full CMS files and local databases remain excluded from git.

The study uses no patient data. Results are descriptive and must not be presented as insurance advice, causal evidence, or clinical guidance. Premiums are not enrollment weighted or subsidy adjusted. Quality associations do not imply that quality causes price differences.

11 Conclusion

Natural-language analytics evaluation should not stop when SQL executes. This project separates metric routing, compilation, policy validation, execution, strict result contracts, compatible projection, and evidence reporting on a real public insurance dataset. The experiments show a large execution–semantics gap and demonstrate that even oracle metric routing leaves substantial compiler error under lexical shift.

The current evidence supports a conservative claim: explicit metric and evidence stages make benchmark failures more traceable and, under benchmark-specific controlled conditions, achieve higher result agreement than the evaluated subscription Codex-to-SQL pilot. It does not support general healthcare reasoning, universal hallucination reduction, or production readiness. The next publication milestone is an independently human-authored or validated locked test set, second-reviewed reference SQL, and completed two-reviewer qualitative evaluation.

References

[1] T. Yu et al. "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task." *EMNLP*, 2018.

[2] J. Li et al. "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs." arXiv:2305.03111, 2023.

[3] R. Zhong, T. Yu, and D. Klein. "Semantic Evaluation for Text-to-SQL with Distilled Test Suites." *EMNLP*, 2020.

[4] A. Hazoom et al. "Evaluating Cross-Domain Text-to-SQL Models and Benchmarks." *EMNLP*, 2023.

[5] P. Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*, 2020.

[6] Centers for Medicare & Medicaid Services. "Health Insurance Exchange Public Use Files." `https://www.cms.gov/marketplace/resources/data/public-use-files`

[7] Centers for Medicare & Medicaid Services. "2026 Health Insurance Exchanges: Public Use Files FAQs." 2026.